Tab 1

01_dataexploration notebook:

Gemini was used mainly as an AI tool to walk through this project and help with coding and Debugging. It navigated through the project for a better understanding.

USED AI FOR:

1. Provided instructions and installation instructions for accessing the COCO 2017 dataset.
2. Assisted in solving the disk space issue because the COCO images file was over 18GB, and a large file was unable to be processed on the free tier of Colab.

1. Download Only the Annotations

This fetches the text files containing the captions and the image URLs, unzips them into your repo structure, and immediately deletes the zip file to save space.

Python
```
# Create the necessary directories
!mkdir -p data/raw/
!mkdir -p data/processed/images/train

# Download the tiny annotations file directly from COCO
!wget -q http://images.cocodataset.org/annotations/annotations_trainval2017.zip

# Unzip and immediately remove the zip to save disk space
!unzip -q annotations_trainval2017.zip -d data/raw/
!rm annotations_trainval2017.zip
```

3. Provided a quality control checklist to make sure the dataset is perfect for model training.

Quality Control Checklist

Before moving to the **Preprocessing Pipeline**, ask your team (specifically **Jaqueline** and **Blessing**, who are in charge of structure and review) to verify these three things:

- **Complete Sentences:** Are the labels actual sentences like "A child in a red jacket" rather than just tags like "child, red, jacket"?
- **Image-Label Alignment:** Does the caption actually describe the image? (Ensures no "misinterpretation" by the model later)
- **Primary Metric Ready:** Since **CIDEr** is your primary metric, notice how it rewards specific language. Do your captions have "discriminative" words, or are they all "a picture of a thing"?

KEY LEARNINGS:

I learned how to import Kaggle datasets on Colab, how to save disk space, and work through our project without spending money on premium features.

CODE ATTRIBUTIONS:
60% Student, 40%AI

02_model_training.ipynb

02_model_training.ipynb

Tool: Claude (Anthropic)

Summary: Claude was mainly used for debugging errors, helping set up cells, and getting suggestions on risk mitigations.

What AI did:

1. Provided installation instructions for the COCO dataset:
    - Walked us through how to download the COCO 2017 annotations and val2017 images directly from the official COCO website
    - Explained how to unzip the files and set up the correct folder structure in Colab
    - Suggested deleting the zip files immediately after extraction to save disk space since the full dataset is large
    - Helped us figure out that val2017 was the right split to use given our free Colab storage limits
2. Helped with debugging specific cells:
    - Cell 4 — the download kept failing because the zip file path was wrong. AI helped fix the urllib.request download function
    - Cell 5 — kept throwing a FileNotFoundError because Kaggle unzipped the dataset into a weird nested folder structure. AI helped us figure out the correct paths and hardcode them
    - Cell 6 — transforms wasn't defined because torchvision wasn't imported. AI caught the missing import and added it to Cell 1 and Cell 2
    - Cell 9 — the training loop was crashing because pin_memory only works with GPU. AI explained we could ignore the warning on CPU
    - Cell 12 — the CIDEr metric kept crashing the output. Turns out it returns an array instead of a regular number. AI suggested a safe_round() fix that handled it
    - Recovery cell — the model wouldn't load from Drive and threw an HFValidationError. AI caught that we needed to wrap the path in str() before passing it to from_pretrained()
3. Helped with reproducibility and visualization:
    - Suggested setting a random seed (SEED=42) across numpy, Python, and PyTorch in Cell 2 so our results stay consistent every run
    - Helped write the loss curve plot in Cell 10 so we could actually see how the model was improving across epochs
    - Suggested saving the loss curve straight to Google Drive so we'd have it ready for the results folder and slides
4. Helped with risk mitigations:
    - Suggested adding data augmentation in Cell 6 — random flips, color jitter, and slight rotation — applied to training images only so the model doesn't just memorize what it saw

    - Suggested early stopping in Cell 9 with a patience of 2 so training automatically stops if the model stops getting better instead of wasting time and GPU
    - Suggested persistent_workers=True in Cell 8 to keep the DataLoader workers running between epochs which sped things up a lot
5. Helped with hyperparameters:
    - Suggested 5e-5 as the learning rate so we don't accidentally overwrite everything BLIP already learned
    - Recommended gradient clipping at 1.0 to stop the gradients from going crazy during backpropagation
    - Suggested a linear warmup over 100 steps to ease into training instead of jumping straight to full speed
    - Recommended AdamW over regular Adam because it handles weight decay better during fine-tuning
    - Helped us think through beam width and how it affects speed vs caption quality
6. Helped with prompt engineering:
    - Suggested trying "a photo showing" as a prompt to nudge BLIP toward more natural captions
    - Explained that BLIP was originally trained on captions that start with similar phrases so the prompt just fits naturally
    - Told us to test with and without it and compare the scores — which we did
    - The prompt ended up improving BLEU-4 by 10%, METEOR by 4%, and ROUGE-L by 2% without any extra training
    - AI explained this is called inference-time prompt engineering — better results without touching the model at all
7. Helped with Google Drive integration:
    - Gave us code to save the best checkpoint to Drive after each epoch using model.save_pretrained() so nothing gets lost if Colab cuts out
    - Suggested only saving when validation loss improves so Drive always has the best version
    - Helped set up the recovery cell so we could pick up right where we left off after a disconnect

Key learnings:

- BLIP works really well even after just 2 epochs of fine-tuning thanks to how much it already learned from 129 million image-caption pairs
- Saving to Drive every epoch saved us a lot of headaches when Colab kept disconnecting
- Early stopping and augmentation made a noticeable difference on our small dataset
- Just adding a prompt improved our scores without any retraining which was a nice surprise
- Debugging honestly took just as long as writing the code

Code attribution: 65% Student / 35% AI

04_demo.ipynb

Demo Notebook (04_demo.ipynb)

Tool: Claude (Anthropic)

Summary: Claude was used to help set up the environment, debug errors, and improve the caption output.

What AI did:

1. Helped set up the demo notebook:
    - Helped structure the notebook so it loads the fine-tuned model from Hugging Face instead of Google Drive so anyone can run it
    - Suggested removing the Google Drive mount cell since the model loads directly from Hugging Face
    - Helped set up the file upload cell so users can upload any image and get a caption instantly
    - Suggested adding a batch demo cell in Cell 6 so multiple images can be captioned and displayed in a grid at once
2. Helped with debugging specific cells:
    - Cell 4 — the model kept throwing an HFValidationError when loading from Drive. AI caught that the path needed to be a string not a Path object and suggested switching to Hugging Face to avoid the issue entirely
    - Cell 5 — the uploaded image wasn't displaying correctly inline. AI suggested using matplotlib instead of IPython display for a cleaner look
    - Cell 6 — the batch grid was crashing when only one image was uploaded. AI caught that axes needed to be wrapped in a list when n=1
3. Helped improve the demo output:
    - Suggested generating top 3 captions using num_return_sequences=3 with beam search so the demo shows the model considered multiple options
    - Added a generation timer using time.time() so the demo shows how fast the model runs
    - Suggested adding length_penalty and repetition_penalty to encourage longer more specific captions
    - Suggested the "a photo showing" prompt to make captions more natural and consistent across different image types
4. Helped clean up the notebook for GitHub:
    - Suggested clearing all outputs before downloading to remove widget metadata that was causing GitHub to throw an invalid notebook error
    - Provided a Terminal script to clean the notebook metadata so it renders correctly on GitHub without losing any outputs

Key learnings:

- Hosting the model on Hugging Face makes the demo way more accessible — anyone can run it with no setup
- Beam search with width 4 and top 3 sequences makes the demo look a lot more impressive than just one caption

- Prompt engineering was a simple but effective way to improve caption quality without any retraining
- Pushing to GitHub seems simple but little things like widget metadata can break how your notebook looks — always clean before uploading

Code attribution: 60% Student / 40% AI

pretrained model links

BLIP-2 Main Repository: [Salesforce/blip2-opt-2.7b - Hugging Face]